End-to-End Data Management Pipeline for a Recommendation System

- Execution screen Shots
- Final Report summary
- Github repository Link

Execution screen Shots

1. python3 -m src.pipeline

```
dn@Deepaks-MacBook-Pro DM4ML_recommendation-pipeline % python3 -m src.pipeline
Movie recommendation pipeline
Started UTC: 2026-11-08T07:11:35.375212+00:00
START ingestion
COMPLETE ingestion
START validation
COMPLETE validation
START preparation
COMPLETE preparation
START transformation
COMPLETE transformation
START feature_management
COMPLETE feature_management
START model_training
COMPLETE model_training
START model_evaluation
COMPLETE model_evaluation
START reporting
COMPLETE reporting
Finished UTC: 2026-11-08T07:11:37.217786+00:00
STATUS: SUCCESS
Movies in catalog: 5000
              model  precision_at_5  recall_at_5  ndcg_at_5  coverage  diversity  evaluated_users
         popularity          0.0144        0.072   0.040024    0.0014      0.976            500.0
content_based_genre          0.2000        1.000   1.000000    0.0390      0.000            500.0
dn@Deepaks-MacBook-Pro DM4ML_recommendation-pipeline %
```

2. python3 dags/workflow.py

```
dn@Deepaks-MacBook-Pro DM4ML_recommendation-pipeline % python3 dags/workflow.py
Movie recommendation pipeline
Started UTC: 2026-11-08T07:36:18.352024+00:00
START ingestion
COMPLETE ingestion
START validation
COMPLETE validation
START preparation
COMPLETE preparation
START transformation
COMPLETE transformation
START feature_management
COMPLETE feature_management
START model_training
COMPLETE model_training
START model_evaluation
COMPLETE model_evaluation
START reporting
COMPLETE reporting
Finished UTC: 2026-11-08T07:36:20.200104+00:00
STATUS: SUCCESS
Movies in catalog: 5000
              model  precision_at_5  recall_at_5  ndcg_at_5  coverage  diversity  evaluated_users
         popularity          0.0144        0.072   0.040024    0.0014      0.976            500.0
content_based_genre          0.2000        1.000   1.000000    0.0390      0.000            500.0
dn@Deepaks-MacBook-Pro DM4ML_recommendation-pipeline %
```

Final Report summary:

Movie Recommendation Data Pipeline ? Final Report

Design and data management

The project creates a deterministic, synthetic English-language movie catalog of exactly 5,000 records. Every synthetic movie is labelled English (language_code=en); titles and descriptions are generated templates, not scraped real-world movie records. It also generates 500 synthetic user profiles and 16,000 valid ratings. Deliberately malformed interaction records exercise the validation and quarantine path. No external dataset is downloaded.

The modular workflow validates required fields, duplicates, types, 1?5 rating bounds, timestamps, English-language catalog policy, and foreign keys. Invalid interactions and source profile/movie rows are logged separately. Preparation fills missing ages, normalizes fields, and holds out at least 1 latest event(s) per user; all events tied at the cutoff timestamp are held out together. Training-only transformations produce activity summaries, movie popularity, a user-item matrix, and reusable user/movie/interaction features.

Models and evaluation

The popularity baseline ranks movies by training interaction count times mean rating. The personalized content-based model learns each user's genre preferences from their training ratings and recommends unseen movies in preferred genres, using popularity as a tie-breaker and cold-start fallback. To make the synthetic offline task testable, users rate four of the five reserved movies in their preferred genre in training and the remaining one is held out as a future positive event. Both recommenders rank rather than predict explicit rating values, so RMSE and MAE do not apply. Ratings >= 4 in the holdout are relevant; K=5. All test users contribute to the averaged ranking metrics. Results from this reproducible run:

Model | Precision@5 | Recall@5 | NDCG@5 | Coverage | Diversity
popularity | 0.014 | 0.072 | 0.040 | 0.001 | 0.976
content_based_genre | 0.200 | 1.000 | 1.000 | 0.039 | 0.000

Orchestration, tracking, and limitations

dags/workflow.py runs the dependency-checked stages; dags/run_daily.sh supports cron scheduling. The JSON run record tracks configuration, metrics, and artifact paths without an external tracking service. Tabular data and features are saved as CSV, run metadata and recommendations as JSON, and evaluation reports as Markdown and PDF.

The controlled synthetic preferences make this suitable for testing data

Github repository Link: https://github.com/2025ag05345git/DM4ML_recommendation-pipeline.git